
ML Interview Questions & Answers

মেশিন লার্নিং ইন্টারভিউ প্রশ্ন ও উত্তর

A detailed, step-by-step bilingual guide (English + Bangla)

ধাপে ধাপে বিস্তারিত দ্বিভাষিক গাইড (ইংরেজি + বাংলা)

Contents সূচিপত্র

1–10 Machine Learning Fundamentals	মেশিন লার্নিং বেসিক্স
11–20 Modeling and Evaluation	মডেলিং ও মূল্যায়ন
21–30 Practical ML Projects	প্র্যাকটিক্যাল এমএল প্রজেক্ট

Part 1: Machine Learning Fundamentals

পার্ট ১: মেশিন লার্নিং বেসিক্স

1 What is the difference between supervised, unsupervised, and reinforcement learning?

সুপারভাইজড, আনসুপারভাইজড এবং রিইনফোর্সমেন্ট লার্নিং-এর মধ্যে পার্থক্য কী?

ENGLISH

1. **Supervised learning:** The model learns from labeled data — every input has a known correct output (e.g., house features → price, email → spam/not spam). The goal is to learn a mapping function that generalizes to new inputs. Examples: regression, classification.
2. **Unsupervised learning:** The data has no labels. The model tries to find hidden structure — grouping similar points (clustering), reducing dimensions (PCA), or detecting anomalies.
3. **Reinforcement learning:** An agent interacts with an environment, takes actions, and receives rewards or penalties. It learns a policy that maximizes cumulative reward over time through trial and error, rather than from a fixed labeled dataset.

Key distinction: supervised = learn from answers, unsupervised = discover patterns without answers, reinforcement = learn from consequences of actions.

বাংলা

1. **সুপারভাইজড লার্নিং:** এখানে ডেটাতে লেবেল (সঠিক উত্তর) দেওয়া থাকে — যেমন বাড়ির তথ্য থেকে দাম, বা ইমেইল স্প্যাম কিনা। মডেল ইনপুট থেকে আউটপুটের একটি ম্যাপিং শেখে, যা নতুন ডেটাতেও কাজ করবে।
উদাহরণ: রিগ্রেশন, ক্লাসিফিকেশন।
2. **আনসুপারভাইজড লার্নিং:** ডেটাতে কোনো লেবেল থাকে না। মডেল নিজে থেকে ডেটার মধ্যে লুকানো প্যাটার্ন খুঁজে বের করে — যেমন একই রকম ডেটা একসাথে গ্রুপ করা (ক্লাস্টারিং), ডাইমেনশন কমানো (PCA), বা অস্বাভাবিক ডেটা খুঁজে বের করা।
3. **রিইনফোর্সমেন্ট লার্নিং:** এখানে একটি এজেন্ট পরিবেশের সাথে ইন্টারঅ্যাক্ট করে, বিভিন্ন অ্যাকশন নেয় এবং তার বিনিময়ে রিওয়ার্ড বা শাস্তি পায়। এজেন্ট বারবার চেষ্টা করে এমন একটি পলিসি শেখে যা দীর্ঘমেয়াদে সবচেয়ে বেশি রিওয়ার্ড দেয়।

মূল পার্থক্য: সুপারভাইজড = উত্তর দেখে শেখা, আনসুপারভাইজড = উত্তর ছাড়াই প্যাটার্ন খোঁজা, রিইনফোর্সমেন্ট = অ্যাকশনের ফলাফল থেকে শেখা।

2 What is the difference between overfitting and underfitting? How do you identify each one?

ওভারফিটিং ও আন্ডারফিটিং-এর মধ্যে পার্থক্য কী? কীভাবে এগুলো শনাক্ত করবেন?

ENGLISH

1. **Overfitting:** The model learns the training data too well, including noise and random fluctuations. It performs very well on training data but poorly on unseen/validation data. Sign: low training error, high validation error (large gap).
2. **Underfitting:** The model is too simple to capture the underlying pattern. It performs poorly on both training and validation data. Sign: high training error and high validation error, both close in value.
3. **How to identify:** Plot training vs. validation loss curves. A widening gap = overfitting. Both curves flat and high = underfitting.
4. **Fixes:** Overfitting → regularization, more data, dropout, simpler model, early stopping. Underfitting → more complex model, more features, train longer, reduce regularization.

বাংলা

1. **ওভারফিটিং:** মডেল ট্রেনিং ডেটা এত বেশি ভালোভাবে মুখস্থ করে ফেলে যে তার মধ্যে থাকা নয়েজ (noise) পর্যন্ত শিখে ফেলে। ফলে ট্রেনিং ডেটাতে খুব ভালো পারফর্ম করলেও নতুন/ভ্যালিডেশন ডেটাতে খারাপ করে। লক্ষণ: ট্রেনিং error কম, ভ্যালিডেশন error বেশি — বড় গ্যাপ।
2. **আন্ডারফিটিং:** মডেল এতটাই সাধারণ (simple) যে ডেটার আসল প্যাটার্নই ধরতে পারে না। ট্রেনিং ও ভ্যালিডেশন — দুই জায়গাতেই খারাপ করে। লক্ষণ: দুই error-ই বেশি এবং কাছাকাছি।
3. **কীভাবে চেনা যায়:** ট্রেনিং বনাম ভ্যালিডেশন লস কার্ভ প্লট করুন। গ্যাপ বাড়তে থাকলে ওভারফিটিং, দুটোই উঁচু ও সমান থাকলে আন্ডারফিটিং।
4. **সমাধান:** ওভারফিটিং-এর জন্য — রেগুলারাইজেশন, বেশি ডেটা, ড্রপআউট, সহজ মডেল, আর্লি স্টপিং। আন্ডারফিটিং-এর জন্য — জটিল মডেল, বেশি ফিচার, বেশি ট্রেনিং, রেগুলারাইজেশন কমানো।

3 Explain the bias-variance trade-off.

বায়াস-ভ্যারিয়েন্স ট্রেড-অফ ব্যাখ্যা করুন।

ENGLISH

1. **Bias:** Error from overly simplistic assumptions in the model. High bias means the model underfits — it misses relevant patterns (e.g., using a linear model for non-linear data).
2. **Variance:** Error from the model being too sensitive to small fluctuations in training data. High variance means the model overfits — it captures noise instead of signal.
3. **Trade-off:** Total error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible error}$. Simple models → high bias, low variance. Complex models → low bias, high variance. The goal is to find the sweet spot that minimizes total error on unseen data.
4. **Practical control:** regularization, model complexity, ensembling (e.g., bagging reduces variance, boosting reduces bias), and more training data (reduces variance).

বাংলা

1. **বায়াস (Bias):** মডেল খুব সরল অনুমান করলে যে error হয়। বায়াস বেশি হলে মডেল আন্ডারফিট করে — আসল প্যাটার্ন ধরতে পারে না (যেমন নন-লিনিয়ার ডেটার জন্য লিনিয়ার মডেল ব্যবহার করা)।
 2. **ভ্যারিয়েন্স (Variance):** ট্রেনিং ডেটার সামান্য পরিবর্তনেও মডেল খুব বেশি সংবেদনশীল হলে যে error হয়। ভ্যারিয়েন্স বেশি হলে মডেল ওভারফিট করে — সিগন্যালের বদলে নয়েজ শিখে ফেলে।
 3. **ট্রেড-অফ:** মোট Error = $\text{Bias}^2 + \text{Variance} + \text{অপরিহার্য error}$ । সহজ মডেলে বায়াস বেশি, ভ্যারিয়েন্স কম। জটিল মডেলে বায়াস কম, ভ্যারিয়েন্স বেশি। লক্ষ্য হলো এমন একটা ভারসাম্য খুঁজে বের করা যেখানে নতুন ডেটাতে মোট error সবচেয়ে কম হয়।
 4. **নিয়ন্ত্রণের উপায়:** রেগুলারাইজেশন, মডেল কমপ্লেক্সিটি ঠিক করা, এনসেম্বল (bagging দিয়ে ভ্যারিয়েন্স কমে, boosting দিয়ে বায়াস কমে), এবং বেশি ট্রেনিং ডেটা (ভ্যারিয়েন্স কমায়ে)।
-

4 What is the difference between parameters and hyperparameters? Give examples.

প্যারামিটার এবং হাইপারপ্যারামিটারের মধ্যে পার্থক্য কী? উদাহরণ দিন।

ENGLISH

1. **Parameters:** Values the model learns automatically from training data during the training process. They define the learned relationship. Examples: weights and biases in a neural network, coefficients in linear regression, split points in a decision tree.
2. **Hyperparameters:** Configuration values set manually before training begins; they control how the training process happens. They are not learned from data (unless you use tuning methods). Examples: learning rate, number of layers, number of trees, regularization strength (λ/α), k in k -NN, batch size.
3. **Key distinction:** parameters come out of training; hyperparameters go into training. Hyperparameters are usually tuned using validation data or techniques like grid search, random search, or Bayesian optimization.

বাংলা

1. **প্যারামিটার:** ট্রেনিং প্রসেসের মধ্যে মডেল নিজে থেকে যে ভ্যালুগুলো শেখে। উদাহরণ: নিউরাল নেটওয়ার্কের weights ও biases, লিনিয়ার রিগ্রেশনের coefficients, ডিসিশন ট্রি split points।
2. **হাইপারপ্যারামিটার:** ট্রেনিং শুরুর আগেই আমরা নিজেরা যে সেটিংসগুলো ঠিক করে দিই, যা ট্রেনিং কীভাবে হবে তা নিয়ন্ত্রণ করে। উদাহরণ: learning rate, layer সংখ্যা, tree সংখ্যা, regularization strength (λ/α), k -NN-এর k , batch size।
3. **মূল পার্থক্য:** প্যারামিটার ট্রেনিং থেকে "বের হয়ে আসে", হাইপারপ্যারামিটার ট্রেনিং-এ "চুকিয়ে দেওয়া হয়"। হাইপারপ্যারামিটার সাধারণত ভ্যালিডেশন ডেটা বা grid search, random search, Bayesian optimization-এর মাধ্যমে টিউন করা হয়।

5 Why do we split data into training, validation, and test sets?

কেন আমরা ডেটাকে ট্রেনিং, ভ্যালিডেশন এবং টেস্ট সেটে ভাগ করি?

ENGLISH

1. **Training set:** Used to actually fit the model — the model learns parameters from this data.
2. **Validation set:** Used during development to tune hyperparameters and select the best model/architecture, without touching the test set. Prevents leakage of test information into model choices.
3. **Test set:** Used only once, at the very end, to get an unbiased estimate of how the model will perform on completely unseen, real-world data.
4. **Why separate:** If we tune using the same data we test on, we get an overly optimistic, biased performance estimate — the model may just be "memorizing" that set rather than generalizing.

বাংলা

1. **ট্রেনিং সেট:** এই ডেটা দিয়েই মডেল আসলে ফিট (train) করা হয় — মডেল এখান থেকেই প্যারামিটার শেখে।
 2. **ভ্যালিডেশন সেট:** ডেভেলপমেন্টের সময় হাইপারপ্যারামিটার টিউন করতে এবং সেরা মডেল/আর্কিটেকচার বেছে নিতে ব্যবহার হয়, টেস্ট সেট স্পর্শ না করেই। এতে টেস্ট ডেটার তথ্য মডেল নির্বাচনে "leak" হয় না।
 3. **টেস্ট সেট:** একদম শেষে, একবারই ব্যবহার করা হয় — যাতে বোঝা যায় মডেল সম্পূর্ণ অদেখা, বাস্তব ডেটাতে কেমন পারফর্ম করবে।
 4. **আলাদা করার কারণ:** একই ডেটা দিয়ে টিউন এবং টেস্ট করলে পারফরম্যান্স estimate অতিরিক্ত ভালো (biased) দেখাবে — মডেল সেই ডেটা "মুখস্থ" করে ফেলতে পারে, generalize না করেই।
-

6 What is cross-validation, and when would you use it?

ক্রস-ভ্যালিডেশন কী, এবং কখন এটি ব্যবহার করবেন?

ENGLISH

1. **Definition:** Cross-validation is a technique to evaluate model performance more reliably by splitting data into multiple "folds" and training/testing on different combinations, instead of relying on a single train/validation split.
2. **k-fold CV:** Split data into k equal parts. Train on k-1 parts, validate on the remaining part. Repeat k times, each time with a different validation fold. Average the results.
3. **Why use it:** Gives a more robust estimate of model performance, reduces the risk that results depend on one lucky/unlucky split, and makes better use of limited data.
4. **When to use:** Especially useful with small-to-medium datasets, for hyperparameter tuning, and for comparing models fairly. For time-series data, use time-based splits (e.g., forward-chaining CV) instead of random folds, to avoid leakage from the future.

বাংলা

1. **সংজ্ঞা:** ক্রস-ভ্যালিডেশন হলো মডেলের পারফরম্যান্স আরও নির্ভরযোগ্যভাবে যাচাই করার একটি পদ্ধতি, যেখানে ডেটাকে একাধিক "fold"-এ ভাগ করে বিভিন্ন কন্সিগনেশনে ট্রেন/টেস্ট করা হয়, শুধু একটিমাত্র split-এর উপর নির্ভর না করে।
2. **k-fold CV:** ডেটাকে k সমান ভাগে ভাগ করুন। k-1 ভাগ দিয়ে ট্রেন করুন, বাকি একটি ভাগ দিয়ে ভ্যালিডেট করুন। এটি k বার করুন, প্রতিবার ভিন্ন ভাগকে ভ্যালিডেশনের জন্য ব্যবহার করে। ফলাফলের গড় নিন।
3. **কেন ব্যবহার করবেন:** মডেলের পারফরম্যান্সের আরও নির্ভরযোগ্য estimate পাওয়া যায়, একটি নির্দিষ্ট split-এর উপর নির্ভরতা কমে, এবং সীমিত ডেটাও ভালোভাবে ব্যবহার করা যায়।
4. **কখন ব্যবহার করবেন:** ছোট বা মাঝারি ডেটাসেটে, হাইপারপ্যারামিটার টিউনিং-এ, এবং মডেল তুলনা করার সময় বিশেষভাবে কার্যকর। টাইম-সিরিজ ডেটার জন্য random fold না করে সময়-ভিত্তিক split (forward-chaining CV) ব্যবহার করতে হয়, যাতে ভবিষ্যতের ডেটা অতীতে "leak" না করে।

7 What is data leakage? Give two practical examples.

ডেটা লিকেজ কী? দুটি বাস্তব উদাহরণ দিন।

ENGLISH

1. **Definition:** Data leakage happens when information from outside the training data (often from the target or future/test data) unintentionally influences the model, making it perform unrealistically well during development but fail in production.
2. **Example 1 — Target leakage:** Predicting loan default using a feature like "days_since_last_payment" that is only recorded after default status is known. The model "cheats" using information that wouldn't be available at prediction time.
3. **Example 2 — Train/test contamination:** Scaling or imputing missing values using statistics (mean, std) computed from the entire dataset (train + test) before splitting. The test set's information leaks into the training process, causing over-optimistic evaluation.
4. **Prevention:** Always fit preprocessing steps only on training data; carefully check that features would realistically be available at prediction time; use time-aware splits for temporal data.

বাংলা

1. **সংজ্ঞা:** ডেটা লিকেজ হয় যখন ট্রেনিং ডেটার বাইরের তথ্য (প্রায়ই টার্গেট বা ভবিষ্যতের/টেস্ট ডেটা থেকে) অনিচ্ছাকৃতভাবে মডেলে প্রভাব ফেলে — ফলে ডেভেলপমেন্টের সময় মডেল অস্বাভাবিক ভালো পারফর্ম করে কিন্তু প্রোডাকশনে গিয়ে ব্যর্থ হয়।
2. **উদাহরণ ১ — টার্গেট লিকেজ:** লোন ডিফল্ট প্রেডিক্ট করতে "days_since_last_payment"-এর মতো ফিচার ব্যবহার করা, যা শুধুমাত্র ডিফল্ট হওয়ার পরেই রেকর্ড হয়। মডেল এমন তথ্য ব্যবহার করে "চিট" করে যা প্রেডিকশনের সময় আসলে পাওয়া যাবে না।
3. **উদাহরণ ২ — ট্রেনিং/টেস্ট কন্টামিনেশন:** split করার আগেই পুরো ডেটাসেট (ট্রেনিং + টেস্ট) থেকে গণনা করা mean/std দিয়ে স্কেলিং বা মিসিং ভ্যালু ফিল করা। এতে টেস্ট সেটের তথ্য ট্রেনিং প্রসেসে "leak" হয়ে যায়, ফলে মূল্যায়ন অতিরিক্ত ভালো দেখায়।
4. **প্রতিরোধ:** সবসময় preprocessing শুধুমাত্র ট্রেনিং ডেটার উপর fit করুন; ভালোভাবে যাচাই করুন কোনো ফিচার প্রেডিকশনের সময় বাস্তবেই পাওয়া যাবে কিনা; টাইম-সিরিজ ডেটার জন্য সময়-সচেতন split ব্যবহার করুন।

8 Why is feature scaling required for some models but not for tree-based models?

কিছু মডেলের জন্য ফিচার স্কেলিং প্রয়োজন কিন্তু ডিভাইসড মডেলের জন্য কেন প্রয়োজন হয় না?

ENGLISH

1. **Distance/gradient-based models need scaling:** Algorithms like k-NN, SVM, k-means, and linear/logistic regression (with gradient descent) rely on distances or gradient magnitudes. If features have very different scales (e.g., income in thousands vs. age in tens), features with larger ranges dominate the distance calculation or gradient updates, biasing the model.
2. **Tree-based models don't need scaling:** Decision trees, Random Forest, XGBoost, LightGBM split data based on threshold comparisons on one feature at a time (e.g., "is age > 30?"). The relative order of values matters, not their scale — a monotonic transformation (like scaling) doesn't change where the best split point falls.
3. **Practical takeaway:** Always scale features for KNN, SVM, PCA, linear/logistic regression, and neural networks. Scaling is not necessary for tree-based models like Random Forest, Gradient Boosting, XGBoost, etc.

বাংলা

1. **ডিসট্যান্স/গ্র্যাডিয়েন্ট-বেসড মডেলে স্কেলিং প্রয়োজন:** k-NN, SVM, k-means, এবং লিনিয়ার/লজিস্টিক রিগ্রেশন (gradient descent সহ)-এর মতো অ্যালগরিদম distance বা gradient-এর উপর নির্ভর করে। ফিচারগুলোর স্কেল খুব আলাদা হলে (যেমন income হাজারে, বয়স দশকে), বড় রেঞ্জের ফিচার distance calculation বা gradient update-এ প্রাধান্য পেয়ে যায়, যা মডেলকে বায়াসড করে।
2. **ডিভাইসড মডেলে স্কেলিং লাগে না:** Decision Tree, Random Forest, XGBoost, LightGBM একটি ফিচারের উপর থ্রেশহোল্ড তুলনা করে ডেটা split করে (যেমন "বয়স > ৩০ কিনা")। এখানে ভ্যালুগুলোর আপেক্ষিক ক্রম গুরুত্বপূর্ণ, স্কেল নয় — স্কেলিং-এর মতো একটি monotonic transformation সেটা split পয়েন্ট বদলায় না।
3. **প্র্যাকটিক্যাল উপসংহার:** KNN, SVM, PCA, লিনিয়ার/লজিস্টিক রিগ্রেশন, এবং নিউরাল নেটওয়ার্কের জন্য সবসময় স্কেলিং করুন। কিন্তু Random Forest, Gradient Boosting, XGBoost-এর মতো ডিভাইসড মডেলের জন্য স্কেলিং করার দরকার নেই।

9 What is the difference between normalization and standardization?

নরমালাইজেশন এবং স্ট্যান্ডার্ডাইজেশনের মধ্যে পার্থক্য কী?

ENGLISH

1. **Normalization (Min-Max scaling):** Rescales values into a fixed range, usually $[0, 1]$, using formula $(x - \min) / (\max - \min)$. Sensitive to outliers, since min/max can be extreme values.
2. **Standardization (Z-score scaling):** Rescales values to have mean 0 and standard deviation 1, using formula $(x - \text{mean}) / \text{std}$. Does not bound values to a fixed range, but is more robust to outliers than min-max scaling.
3. **When to use which:** Use normalization when you need bounded ranges (e.g., image pixel values, neural network inputs where activation functions expect bounded input). Use standardization when the data roughly follows a Gaussian-like distribution or is used in algorithms like PCA, logistic regression, and SVM that assume centered data.

বাংলা

1. **নরমালাইজেশন (Min-Max স্কেলিং):** ভ্যালুগুলোকে একটি নির্দিষ্ট রেঞ্জে, সাধারণত $[0, 1]$ -এ নিয়ে আসে, সূত্র: $(x - \min) / (\max - \min)$ । আউটলায়ারের প্রতি সংবেদনশীল, কারণ min/max চরম মান হতে পারে।
2. **স্ট্যান্ডার্ডাইজেশন (Z-score স্কেলিং):** ভ্যালুগুলোকে এমনভাবে রূপান্তর করে যাতে mean হয় 0 এবং standard deviation হয় 1, সূত্র: $(x - \text{mean}) / \text{std}$ । এটি নির্দিষ্ট রেঞ্জে বাঁধা থাকে না, কিন্তু min-max স্কেলিং-এর চেয়ে আউটলায়ারের প্রতি বেশি robust।
3. **কখন কোনটা ব্যবহার করবেন:** বাঁধা রেঞ্জ দরকার হলে (যেমন ইমেজ পিক্সেল ভ্যালু, নিউরাল নেটওয়ার্কের ইনপুট যেখানে activation function বাউন্ডেড ইনপুট আশা করে) নরমালাইজেশন ব্যবহার করুন। ডেটা মোটামুটি Gaussian-এর মতো ডিস্ট্রিউশন অনুসরণ করলে, বা PCA, লজিস্টিক রিগ্রেশন, SVM-এর মতো অ্যালগরিদমে যেখানে centered ডেটা প্রয়োজন, সেখানে স্ট্যান্ডার্ডাইজেশন ব্যবহার করুন।

10 How do L1 and L2 regularization differ?

L1 এবং L2 রেগুলারাইজেশন কীভাবে আলাদা?

ENGLISH

1. **L1 regularization (Lasso):** Adds the sum of absolute values of weights ($\lambda \sum |w|$) to the loss function. Tends to push some weights exactly to zero, effectively performing automatic feature selection and producing sparse models.
2. **L2 regularization (Ridge):** Adds the sum of squared weights ($\lambda \sum w^2$) to the loss function. Shrinks weights toward zero but rarely makes them exactly zero; it spreads the penalty more smoothly, keeping all features but reducing their impact.
3. **Effect on model:** L1 $\rightarrow$ sparse model, good when you suspect only a few features truly matter. L2 $\rightarrow$ keeps all features but reduces overfitting and stabilizes coefficients, good when features are correlated.
4. **Elastic Net:** Combines both L1 and L2 for a balance between sparsity and stability.

বাংলা

1. **L1 রেগুলারাইজেশন (Lasso):** লস ফাংশনে weight-গুলোর absolute value-এর যোগফল ($\lambda \sum |w|$) যোগ করে। এটি অনেক weight-কে একদম শূন্য করে দেওয়ার প্রবণতা রাখে, ফলে স্বয়ংক্রিয়ভাবে ফিচার সিলেকশন হয়ে যায় এবং sparse মডেল তৈরি হয়।
 2. **L2 রেগুলারাইজেশন (Ridge):** লস ফাংশনে weight-গুলোর বর্গের যোগফল ($\lambda \sum w^2$) যোগ করে। এটি weight-গুলোকে শূন্যের দিকে ছোট করে কিন্তু সাধারণত একদম শূন্য করে না; পেনাল্টি সব ফিচারের মধ্যে মসৃণভাবে ছড়িয়ে দেয়, সব ফিচার রেখে দিয়ে তাদের প্রভাব কমায়।
 3. **মডেলে প্রভাব:** L1 $\rightarrow$ sparse মডেল, যখন মনে হয় কিছু ফিচারই আসলে গুরুত্বপূর্ণ। L2 $\rightarrow$ সব ফিচার রেখে দেয় কিন্তু ওভারফিটিং কমায় এবং coefficient স্থিতিশীল করে, ফিচারগুলো correlated হলে ভালো কাজ করে।
 4. **Elastic Net:** sparsity ও স্থিতিশীলতার মধ্যে ভারসাম্যের জন্য L1 ও L2 দুটোই একসাথে ব্যবহার করে।
-

Part 2: Modeling and Evaluation

পার্ট ২: মডেলিং ও মূল্যায়ন

11 When should you use accuracy, precision, recall, and F1-score?

কখন accuracy, precision, recall এবং F1-score ব্যবহার করবেন?

ENGLISH

1. **Accuracy:** (Correct predictions / Total predictions). Good only when classes are balanced. Misleading on imbalanced data (e.g., 95% accuracy is meaningless if 95% of data is one class).
2. **Precision:** Of all predicted positives, how many were actually positive. Use when false positives are costly (e.g., spam detection — you don't want to mark real emails as spam).
3. **Recall:** Of all actual positives, how many were correctly identified. Use when false negatives are costly (e.g., cancer/fraud detection — missing a real case is dangerous).
4. **F1-score:** Harmonic mean of precision and recall. Use when you need a balance between the two, especially on imbalanced datasets, and there's no strong preference for avoiding one error type over the other.

বাংলা

1. **Accuracy:** (সঠিক প্রেডিকশন / মোট প্রেডিকশন)। শুধুমাত্র ক্লাসগুলো ব্যালান্সড থাকলে ভালো কাজ করে। ইমব্যালান্সড ডেটাতে ভুল ধারণা দেয় (যেমন 95% ডেটা এক ক্লাসের হলে 95% accuracy-র কোনো অর্থ নেই)।
2. **Precision:** যত পজিটিভ প্রেডিক্ট করা হয়েছে, তার মধ্যে কতগুলো আসলেই পজিটিভ ছিল। False positive-এর খরচ বেশি হলে ব্যবহার করুন (যেমন স্প্যাম ডিটেকশন — আসল ইমেইলকে স্প্যাম বলা যাবে না)।
3. **Recall:** যত আসল পজিটিভ ছিল, তার মধ্যে কতগুলো সঠিকভাবে ধরা পড়েছে। False negative-এর খরচ বেশি হলে ব্যবহার করুন (যেমন ক্যান্সার/ফ্রড ডিটেকশন — আসল কেস মিস করা বিপজ্জনক)।
4. **F1-score:** Precision ও Recall-এর harmonic mean। যখন দুটোর মধ্যে ভারসাম্য দরকার, বিশেষত ইমব্যালান্সড ডেটাসেটে, এবং একটি error-কে অন্যটির চেয়ে এড়ানোর জোরালো পছন্দ না থাকলে, তখন ব্যবহার করুন।

12 How would you handle a highly imbalanced classification dataset?

অত্যন্ত ইমব্যালান্সড ক্লাসিফিকেশন ডেটাসেট কীভাবে হ্যান্ডেল করবেন?

ENGLISH

1. **Resampling:** Oversample the minority class (e.g., SMOTE, random oversampling) or undersample the majority class.
2. **Class weights:** Assign higher penalty weights to the minority class in the loss function so the model pays more attention to it (available in most libraries via `class_weight='balanced'`).
3. **Better metrics:** Move away from accuracy; use precision, recall, F1, PR-AUC, or ROC-AUC that reflect performance on the minority class.
4. **Algorithm choice:** Tree-based ensembles (Random Forest, XGBoost) with class weighting often perform well; anomaly detection framing can help for extreme imbalance (e.g., fraud with $<1\%$ positive rate).
5. **Threshold tuning:** Adjust the decision threshold instead of using the default 0.5, based on the precision-recall trade-off relevant to the business problem.
6. **Collect more data:** If possible, gather more examples of the minority class.

বাংলা

1. **রিস্যাম্পলিং:** মাইনরিটি ক্লাসকে oversample করুন (যেমন SMOTE, random oversampling) অথবা মেজরিটি ক্লাসকে undersample করুন।
2. **ক্লাস ওয়েট:** লস ফাংশনে মাইনরিটি ক্লাসকে বেশি পেনাল্টি ওয়েট দিন, যাতে মডেল সেদিকে বেশি মনোযোগ দেয় (বেশিরভাগ লাইব্রেরিতে `class_weight='balanced'` দিয়ে পাওয়া যায়)।
3. **ভালো মেট্রিক:** accuracy থেকে সরে এসে precision, recall, F1, PR-AUC, বা ROC-AUC ব্যবহার করুন, যা মাইনরিটি ক্লাসের পারফরম্যান্স ভালোভাবে দেখায়।
4. **অ্যালগরিদম নির্বাচন:** ক্লাস ওয়েটিং সহ স্ট্রিমিং এনসেম্বল (Random Forest, XGBoost) সাধারণত ভালো কাজ করে; অত্যন্ত ইমব্যালান্সের ক্ষেত্রে (যেমন $<1\%$ ফ্রড) anomaly detection হিসেবে সমস্যাটি সাজানো যেতে পারে।
5. **থ্রেশহোল্ড টিউনিং:** ডিফল্ট 0.5-এর বদলে ব্যবসায়িক প্রয়োজন অনুযায়ী precision-recall ট্রেড-অফ বিবেচনা করে থ্রেশহোল্ড ঠিক করুন।
6. **আরও ডেটা সংগ্রহ:** সম্ভব হলে মাইনরিটি ক্লাসের আরও বেশি উদাহরণ সংগ্রহ করুন।

13 What is a confusion matrix, and what can you learn from it?

কনফিউশন ম্যাট্রিক্স কী, এবং এটি থেকে কী জানা যায়?

ENGLISH

1. **Definition:** A table that compares actual labels vs. predicted labels for a classification model, broken into four cells for binary classification: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN).
2. **What it reveals:** Not just overall correctness, but exactly what kind of mistakes the model makes — e.g., is it predicting too many false positives (over-flagging) or too many false negatives (missing cases)?
3. **Derived metrics:** All key classification metrics come from this table — Accuracy = $(TP+TN)/\text{Total}$, Precision = $TP/(TP+FP)$, Recall = $TP/(TP+FN)$, F1 = harmonic mean of precision and recall.
4. **Use case:** Helps diagnose class-specific weaknesses, especially useful for multi-class problems (extended to an $N \times N$ matrix) to see which classes get confused with each other.

বাংলা

1. **সংজ্ঞা:** একটি টেবিল যা ক্লাসিফিকেশন মডেলের আসল লেবেল বনাম প্রেডিক্টেড লেবেল তুলনা করে; বাইনারি ক্লাসিফিকেশনে চারটি ভাগে বিভক্ত: True Positive (TP), True Negative (TN), False Positive (FP), এবং False Negative (FN)।
2. **কী জানা যায়:** শুধু সামগ্রিক সঠিকতা নয়, বরং মডেল ঠিক কোন ধরনের ভুল করছে তা বোঝা যায় — যেমন মডেল কি বেশি false positive দিচ্ছে (অতিরিক্ত ফ্ল্যাগ করছে) নাকি বেশি false negative দিচ্ছে (কেস মিস করছে)?
3. **এখান থেকে যে মেট্রিক পাওয়া যায়:** সব গুরুত্বপূর্ণ ক্লাসিফিকেশন মেট্রিক এই টেবিল থেকেই আসে — Accuracy = $(TP+TN)/\text{মোট}$, Precision = $TP/(TP+FP)$, Recall = $TP/(TP+FN)$, F1 = precision ও recall-এর harmonic mean।
4. **ব্যবহার:** ক্লাস-নির্দিষ্ট দুর্বলতা নির্ণয়ে সাহায্য করে, বিশেষত মাল্টি-ক্লাস সমস্যায় ($N \times N$ ম্যাট্রিক্স বর্ধিত) কোন ক্লাসগুলো একে অপরের সাথে গুলিয়ে যাচ্ছে তা দেখতে সহায়ক।

14 What is probability calibration, and why can it matter in production?

প্রোবাবিলিটি ক্যালিব্রেশন কী, এবং প্রোডাকশনে এটি কেন গুরুত্বপূর্ণ হতে পারে?

ENGLISH

1. **Definition:** A classifier is "calibrated" if its predicted probabilities match real-world observed frequencies — e.g., among all predictions where the model says "70% chance of fraud," roughly 70% should actually be fraud.
2. **Why it matters:** Many models (especially SVM, boosted trees, or neural nets with certain losses) output scores that look like probabilities but aren't well calibrated — they might be systematically over- or under-confident.
3. **Production impact:** If downstream systems use the probability directly (e.g., risk-based pricing, deciding how many fraud cases to route to human review, combining scores from multiple models), miscalibrated probabilities lead to poor business decisions even if the model ranks cases correctly (good AUC).
4. **Fixes:** Platt scaling (logistic regression on model outputs) or isotonic regression, applied on a held-out calibration set.

বাংলা

1. **সংজ্ঞা:** একটি ক্লাসিফায়ার "ক্যালিব্রেটেড" বলা হয় যদি তার প্রেডিক্ট করা প্রোবাবিলিটি বাস্তব ফ্রিকোয়েন্সির সাথে মিলে যায় — যেমন যেসব প্রেডিকশনে মডেল বলছে "70% ফ্রড হওয়ার সম্ভাবনা", সেগুলোর মধ্যে বাস্তবে প্রায় 70%-ই ফ্রড হওয়া উচিত।
2. **কেন গুরুত্বপূর্ণ:** অনেক মডেল (বিশেষত SVM, boosted trees, বা নির্দিষ্ট loss-সহ নিউরাল নেট) এমন স্কোর দেয় যা দেখতে প্রোবাবিলিটির মতো কিন্তু ভালোভাবে ক্যালিব্রেটেড নয় — এগুলো সিস্টেমটিকভাবে অতিরিক্ত বা কম কনফিডেন্ট হতে পারে।
3. **প্রোডাকশনে প্রভাব:** ডাউনস্ট্রিম সিস্টেম যদি সরাসরি প্রোবাবিলিটি ব্যবহার করে (যেমন রিস্ক-বেসড প্রাইসিং, কতগুলো ফ্রড কেস মানুষের রিভিউতে পাঠাতে হবে তা ঠিক করা, একাধিক মডেলের স্কোর একত্র করা), তাহলে ভুলভাবে ক্যালিব্রেটেড প্রোবাবিলিটি খারাপ ব্যবসায়িক সিদ্ধান্তের দিকে নিয়ে যায় — যদিও মডেলের AUC ভালো থাকতে পারে।
4. **সমাধান:** Platt scaling (মডেলের আউটপুটের উপর লজিস্টিক রিগ্রেশন) বা isotonic regression, একটি আলাদা ক্যালিব্রেশন সেটের উপর প্রয়োগ করে।

15 What is the difference between bagging and boosting?

ব্যাগিং এবং বুস্টিং-এর মধ্যে পার্থক্য কী?

ENGLISH

1. **Bagging (Bootstrap Aggregating):** Trains multiple models independently and in parallel on different random bootstrap samples of the data, then averages (regression) or votes (classification) their predictions. Main goal: reduce variance. Example: Random Forest.
2. **Boosting:** Trains models sequentially, where each new model focuses on correcting the errors made by previous models (by reweighting misclassified samples or fitting to residuals). Main goal: reduce bias. Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM.
3. **Key differences:** Bagging trains in parallel and independently; boosting trains sequentially and dependently. Bagging reduces overfitting/variance; boosting can increase risk of overfitting if not regularized, but typically achieves higher accuracy.

বাংলা

1. **ব্যাগিং (Bootstrap Aggregating):** ডেটার বিভিন্ন র‍্যান্ডম bootstrap sample-এ একাধিক মডেল স্বাধীনভাবে ও সমান্তরালে (parallel) ট্রেন করে, তারপর তাদের প্রেডিকশনের গড় (রিগ্রেশন) বা ভোট (ক্লাসিফিকেশন) নেয়। মূল লক্ষ্য: ভ্যারিয়েন্স কমানো। উদাহরণ: Random Forest।
2. **বুস্টিং:** মডেলগুলো ধারাবাহিকভাবে (sequentially) ট্রেন করা হয়, যেখানে প্রতিটি নতুন মডেল আগের মডেলের ভুল সংশোধনের দিকে মনোযোগ দেয় (ভুল ক্লাসিফাইড sample-কে বেশি ওয়েট দিয়ে বা residual-এর উপর ফিট করে)। মূল লক্ষ্য: বায়াস কমানো। উদাহরণ: AdaBoost, Gradient Boosting, XGBoost, LightGBM।
3. **মূল পার্থক্য:** ব্যাগিং সমান্তরাল ও স্বাধীনভাবে ট্রেন করে; বুস্টিং ধারাবাহিক ও নির্ভরশীলভাবে ট্রেন করে। ব্যাগিং ওভারফিটিং/ভ্যারিয়েন্স কমায়; বুস্টিং সঠিকভাবে regularize না করলে ওভারফিটিং-এর ঝুঁকি বাড়াতে পারে, কিন্তু সাধারণত বেশি accuracy দেয়।

16 Compare Random Forest, XGBoost, LightGBM, and CatBoost at a high level.

Random Forest, XGBoost, LightGBM, এবং CatBoost-কে উচ্চ-স্তরে তুলনা করুন।

ENGLISH

1. **Random Forest:** Bagging-based ensemble of decision trees trained independently on bootstrap samples with random feature subsets. Simple, robust, hard to overfit badly, easy to parallelize, but usually slightly less accurate than well-tuned boosting methods.
2. **XGBoost:** Gradient boosting implementation optimized for speed and performance, with built-in regularization (L1/L2), handling of missing values, and tree pruning. Very popular for tabular data competitions; needs more careful hyperparameter tuning than Random Forest.
3. **LightGBM:** Gradient boosting framework that grows trees leaf-wise (rather than level-wise), using histogram-based algorithms for speed. Faster and more memory-efficient than XGBoost on large datasets, especially with many features.
4. **CatBoost:** Gradient boosting library designed to handle categorical features natively without manual encoding, using ordered boosting to reduce prediction shift/overfitting. Often needs less preprocessing and hyperparameter tuning to get good results, especially with many categorical columns.
5. **Choosing:** Random Forest for a strong, low-effort baseline; XGBoost/LightGBM for maximum performance with tuning effort; LightGBM for very large datasets needing speed; CatBoost when there are many categorical features.

বাংলা

1. **Random Forest:** bootstrap sample এবং random feature subset-এ স্বাধীনভাবে ট্রেন করা decision tree-এর bagging-বেসড এনসেম্বল। সহজ, robust, বাজেভাবে ওভারফিট করা কঠিন, সহজেই parallelize করা যায়, কিন্তু সাধারণত ভালোভাবে টিউন করা boosting মেথডের চেয়ে একটু কম accurate।
2. **XGBoost:** গতি ও পারফরম্যান্সের জন্য অপটিমাইজড gradient boosting implementation, built-in regularization (L1/L2), missing value হ্যান্ডলিং, এবং tree pruning সহ। ট্যাবুলার ডেটা প্রতিযোগিতায় খুব জনপ্রিয়; Random Forest-এর চেয়ে বেশি সতর্কভাবে হাইপারপ্যারামিটার টিউনিং দরকার।
3. **LightGBM:** gradient boosting ফ্রেমওয়ার্ক যা tree-কে level-wise না বরং leaf-wise বাড়ায়, গতির জন্য histogram-based অ্যালগরিদম ব্যবহার করে। বড় ডেটাসেটে, বিশেষত অনেক ফিচার থাকলে, XGBoost-এর চেয়ে দ্রুত এবং মেমরি-সাশ্রয়ী।
4. **CatBoost:** ক্যাটাগরিক্যাল ফিচার ম্যানুয়াল এনকোডিং ছাড়াই সরাসরি হ্যান্ডল করার জন্য ডিজাইন করা gradient boosting লাইব্রেরি, prediction shift/ওভারফিটিং কমাতে ordered boosting ব্যবহার করে। অনেক ক্যাটাগরিক্যাল কলাম থাকলে সাধারণত কম প্রিপ্রসেসিং ও টিউনিং-এই ভালো ফলাফল দেয়।
5. **কোনটা বেছে নেবেন:** শক্তিশালী, কম-এফোর্টের বেসলাইনের জন্য Random Forest; টিউনিং এফোর্ট দিয়ে সর্বোচ্চ পারফরম্যান্সের জন্য XGBoost/LightGBM; খুব বড় ডেটাসেটে গতির জন্য LightGBM; অনেক ক্যাটাগরিক্যাল ফিচার থাকলে CatBoost।

17 How does gradient descent work, and what can cause it to fail or converge slowly?

গ্র্যাডিয়েন্ট ডিসেন্ট কীভাবে কাজ করে, এবং কী কারণে এটি ব্যর্থ হতে পারে বা ধীরে converge করতে পারে?

ENGLISH

1. **How it works:** An iterative optimization algorithm that minimizes a loss function by repeatedly updating model parameters in the direction opposite to the gradient (steepest ascent) of the loss with respect to those parameters: $w = w - \alpha \cdot \nabla L(w)$, where α is the learning rate.
2. **Learning rate too high:** Updates overshoot the minimum, causing the loss to oscillate or diverge instead of converging.
3. **Learning rate too low:** Convergence becomes extremely slow, and the model may get stuck in flat regions or take too long to train.
4. **Poor feature scaling:** Unscaled features create an elongated, skewed loss surface, making gradient descent zig-zag and converge slowly.
5. **Local minima / saddle points:** In non-convex loss surfaces (common in deep learning), the optimizer can get stuck in saddle points or poor local minima.
6. **Vanishing/exploding gradients:** In deep networks, gradients can become too small (vanish) or too large (explode) as they propagate backward, stalling or destabilizing training.
7. **Fixes:** Use adaptive optimizers (Adam, RMSProp), learning rate schedules, proper weight initialization, batch normalization, and feature scaling.

বাংলা

1. **কীভাবে কাজ করে:** এটি একটি iterative অপটিমাইজেশন অ্যালগরিদম যা loss function-কে ছোট করে, প্যারামিটারগুলোকে গ্র্যাডিয়েন্টের বিপরীত দিকে বারবার আপডেট করে: $w = w - \alpha \cdot \nabla L(w)$, যেখানে α হলো learning rate।
2. **Learning rate বেশি হলে:** আপডেট মিনিমামকে overshoot করে, ফলে loss কমার বদলে দুলতে থাকে বা diverge করে।
3. **Learning rate কম হলে:** converge হতে অনেক বেশি সময় লাগে, এবং মডেল সমতল অঞ্চলে আটকে থাকতে পারে বা ট্রেনিং-এ বেশি সময় নিতে পারে।
4. **খারাপ ফিচার স্কেলিং:** স্কেল না করা ফিচার একটি লম্বাটে, বাঁকা loss surface তৈরি করে, ফলে গ্র্যাডিয়েন্ট ডিসেন্ট zig-zag করে এবং ধীরে converge করে।
5. **লোকাল মিনিমা / স্যাডল পয়েন্ট:** non-convex loss surface-এ (ডিপ লার্নিংয়ে সাধারণ), অপটিমাইজার saddle point বা খারাপ লোকাল মিনিমায় আটকে যেতে পারে।
6. **Vanishing/exploding গ্র্যাডিয়েন্ট:** ডিপ নেটওয়ার্কে, backward propagation-এর সময় গ্র্যাডিয়েন্ট খুব ছোট (vanish) বা খুব বড় (explode) হয়ে যেতে পারে, যা ট্রেনিং থামিয়ে দেয় বা অস্থির করে তোলে।
7. **সমাধান:** adaptive অপটিমাইজার (Adam, RMSProp), learning rate schedule, সঠিক weight initialization, batch normalization, এবং ফিচার স্কেলিং ব্যবহার করুন।

18 How do you select features for a machine-learning model?

মেশিন লার্নিং মডেলের জন্য কীভাবে ফিচার নির্বাচন করবেন?

ENGLISH

1. **Filter methods:** Rank features using statistical tests independent of the model — correlation with target, chi-square test, mutual information, variance threshold (removing near-constant features).
2. **Wrapper methods:** Use the model itself to evaluate feature subsets — Recursive Feature Elimination (RFE), forward/backward selection. More accurate but computationally expensive.
3. **Embedded methods:** Feature selection happens as part of model training — L1 (Lasso) regularization zeroes out weak features; tree-based models provide feature importance scores directly.
4. **Domain knowledge:** Consult subject-matter expertise to include/exclude features that are known to be meaningful or spurious, and to engineer new informative features.
5. **Practical steps:** Remove highly correlated/redundant features, remove low-variance features, check feature importance from a baseline model, and iteratively validate using cross-validation performance rather than relying on a single metric.

বাংলা

1. **ফিল্টার মেথড:** মডেল ছাড়াই পরিসংখ্যানগত টেস্ট ব্যবহার করে ফিচার র‍্যাঙ্ক করা — টার্গেটের সাথে correlation, chi-square টেস্ট, mutual information, variance threshold (প্রায়-কনস্ট্যান্ট ফিচার বাদ দেওয়া)।
2. **রাপার মেথড:** ফিচার সাবসেট মূল্যায়নে মডেল নিজেই ব্যবহার করা — Recursive Feature Elimination (RFE), forward/backward selection। বেশি সঠিক কিন্তু computationally ব্যয়বহুল।
3. **এমবেডেড মেথড:** মডেল ট্রেনিং-এর অংশ হিসেবেই ফিচার সিলেকশন হয় — L1 (Lasso) রেগুলারাইজেশন দুর্বল ফিচারকে শূন্য করে দেয়; ট্রি-বেসড মডেল সরাসরি feature importance স্কোর দেয়।
4. **ডোমেইন নলেজ:** কোন ফিচার অর্থবহ বা অপ্রাসঙ্গিক তা জানতে বিষয়-বিশেষজ্ঞের পরামর্শ নেওয়া, এবং নতুন তথ্যবহুল ফিচার তৈরি করা।
5. **প্র্যাকটিক্যাল ধাপ:** অতিরিক্ত correlated/redundant ফিচার বাদ দিন, কম variance-এর ফিচার বাদ দিন, একটি বেসলাইন মডেল থেকে feature importance যাচাই করুন, এবং একটিমাত্র মেট্রিকের উপর নির্ভর না করে cross-validation performance দিয়ে বারবার যাচাই করুন।

19 How would you choose a decision threshold for a binary classifier?

বাইনারি ক্লাসিফায়ারের জন্য কীভাবে ডিসিশন থ্রেশহোল্ড বেছে নেবেন?

ENGLISH

1. **Default is not always right:** The default threshold of 0.5 assumes equal cost for false positives and false negatives, which is rarely true in real business problems.
2. **Understand costs:** Identify the real-world cost of a false positive vs. a false negative (e.g., in medical screening, missing a disease/false negative is usually far costlier than a false alarm/false positive).
3. **Use the Precision-Recall or ROC curve:** Plot precision/recall (or TPR/FPR) at different thresholds and pick the point that best matches the business need — e.g., a minimum acceptable recall, or a maximum tolerable false-positive rate.
4. **Optimize a business metric:** Directly optimize threshold for a cost function (e.g., minimize expected monetary loss) or a target metric (e.g., maximize F1, or F-beta if recall/precision should be weighted unevenly).
5. **Validate:** Choose the threshold on a validation set, not the test set, to avoid overfitting the threshold choice itself.

বাংলা

1. **ডিফল্ট সবসময় সঠিক নয়:** ডিফল্ট থ্রেশহোল্ড 0.5 ধরে নেয় যে false positive ও false negative-এর খরচ সমান, যা বাস্তব ব্যবসায়িক সমস্যায় খুব কমই সত্যি হয়।
2. **খরচ বুঝুন:** false positive বনাম false negative-এর বাস্তব খরচ কী তা বের করুন (যেমন মেডিকেল স্ক্রিনিং-এ, রোগ মিস করা/false negative সাধারণত মিথ্যা অ্যালার্ম/false positive-এর চেয়ে অনেক বেশি ব্যয়বহুল)।
3. **Precision-Recall বা ROC কার্ভ ব্যবহার করুন:** বিভিন্ন থ্রেশহোল্ডে precision/recall (বা TPR/FPR) প্লট করুন এবং ব্যবসায়িক প্রয়োজনের সাথে সবচেয়ে মিলে এমন পয়েন্ট বেছে নিন — যেমন একটি ন্যূনতম recall, বা সর্বোচ্চ গ্রহণযোগ্য false-positive rate।
4. **ব্যবসায়িক মেট্রিক অপটিমাইজ করুন:** একটি cost function-এর জন্য সরাসরি থ্রেশহোল্ড অপটিমাইজ করুন (যেমন আর্থিক ক্ষতি কমানো) বা একটি টার্গেট মেট্রিক (যেমন F1 সর্বোচ্চ করা, অথবা recall/precision অসমভাবে গুরুত্বপূর্ণ হলে F-beta)।
5. **যাচাই করুন:** থ্রেশহোল্ড টেস্ট সেটে নয়, ভ্যালিডেশন সেটে ঠিক করুন, যাতে থ্রেশহোল্ড বাছাইটাই ওভারফিট না হয়ে যায়।

20 How do you decide which model to use for a new problem?

নতুন একটি সমস্যার জন্য কোন মডেল ব্যবহার করবেন তা কীভাবে ঠিক করবেন?

ENGLISH

1. **Understand the problem type:** Classification, regression, ranking, clustering, or something else — this narrows the algorithm family immediately.
2. **Consider data characteristics:** Data size (small data favors simpler models; deep learning needs large data), data type (tabular → tree-based models often win; images/text/audio → deep learning), and feature-to-sample ratio.
3. **Consider constraints:** Interpretability requirements (regulated industries may need explainable models like logistic regression or decision trees), latency/inference speed needs, and available compute/infrastructure.
4. **Start simple:** Always establish a simple baseline first (e.g., logistic/linear regression, or a simple tree-based model) before jumping to complex models — this sets a benchmark and often reveals data issues early.
5. **Iterate empirically:** Try a few candidate model families, compare using proper cross-validation and the right metric for the problem, and pick based on evidence rather than assumption. Balance accuracy against interpretability, training/inference cost, and maintainability.

বাংলা

1. **সমস্যার ধরন বুঝুন:** ক্লাসিফিকেশন, রিগ্রেশন, র‍্যাঙ্কিং, ক্লাস্টারিং, নাকি অন্য কিছু — এটি সাথে সাথেই অ্যালগরিদম পরিবারকে সীমিত করে দেয়।
2. **ডেটার বৈশিষ্ট্য বিবেচনা করুন:** ডেটার আকার (ছোট ডেটাতে সহজ মডেল ভালো; ডিপ লার্নিং-এ বড় ডেটা দরকার), ডেটার ধরন (ট্যাবুলার → ট্রি-বেসড মডেল প্রায়ই ভালো করে; ইমেজ/টেক্সট/অডিও → ডিপ লার্নিং), এবং ফিচার-থেকে-স্যাম্পল অনুপাত।
3. **সীমাবদ্ধতা বিবেচনা করুন:** ব্যাখ্যাযোগ্যতার প্রয়োজন (নিয়ন্ত্রিত শিল্পে লজিস্টিক রিগ্রেশন বা ডিসিশন ট্রি মতো explainable মডেল লাগতে পারে), latency/inference গতির প্রয়োজন, এবং উপলব্ধ কম্পিউট/ইনফ্রাস্ট্রাকচার।
4. **সহজ দিয়ে শুরু করুন:** জটিল মডেলে যাওয়ার আগে সবসময় একটি সহজ বেসলাইন তৈরি করুন (যেমন লজিস্টিক/লিনিয়ার রিগ্রেশন, বা একটি সহজ ট্রি-বেসড মডেল) — এটি একটি বেঞ্চমার্ক তৈরি করে এবং প্রায়ই ডেটার সমস্যা আগেই ধরিয়ে দেয়।
5. **অভিজ্ঞতালব্ধভাবে পুনরাবৃত্তি করুন:** কয়েকটি সম্ভাব্য মডেল পরিবার চেষ্টা করুন, সঠিক cross-validation এবং সমস্যার জন্য উপযুক্ত মেট্রিক দিয়ে তুলনা করুন, এবং অনুমানের বদলে প্রমাণের ভিত্তিতে বেছে নিন। accuracy-কে ব্যাখ্যাযোগ্যতা, ট্রেনিং/ইনফারেন্স খরচ এবং maintainability-র সাথে ভারসাম্য রাখুন।

Part 3: Practical ML Projects

পার্ট ৩: প্র্যাকটিক্যাল এমএল প্রজেক্ট

21 Your model performs well offline but poorly in production. How would you investigate?

আপনার মডেল অফলাইনে ভালো পারফর্ম করে কিন্তু প্রোডাকশনে খারাপ করে। কীভাবে তদন্ত করবেন?

ENGLISH

1. **Check for training/serving skew:** Verify that feature computation logic is identical between offline training and online serving (different code paths often cause subtle mismatches).
2. **Check for data leakage in offline evaluation:** Re-verify the offline validation setup didn't have leakage that inflated offline metrics unrealistically.
3. **Check for data drift:** Compare the distribution of production input data against training data — the real-world data may have shifted since training.
4. **Check the pipeline:** Look for bugs in feature engineering, missing value handling, or preprocessing steps that behave differently at inference time (e.g., stale features, joins failing silently).
5. **Check labels/feedback loop:** Confirm production ground-truth labels (if available) are being computed the same way as training labels.
6. **Check for latency/timeout issues:** Sometimes the model gets partial or default input due to system/infra issues rather than a modeling error.
7. **Segment the failure:** Break down errors by user segment, time, or geography to isolate where and when the gap started, which often points directly to the root cause.

বাংলা

1. **ট্রেনিং/সার্ভিং স্কিউ চেক করুন:** নিশ্চিত করুন অফলাইন ট্রেনিং এবং অনলাইন সার্ভিং-এ ফিচার হিসাবের লজিক একই কিনা (আলাদা কোড পাথ প্রায়ই সূক্ষ্ম অমিল তৈরি করে)।
2. **অফলাইন মূল্যায়নে ডেটা লিকেজ চেক করুন:** পুনরায় যাচাই করুন অফলাইন ভ্যালিডেশন সেটআপে এমন কোনো লিকেজ ছিল না যা অফলাইন মেট্রিক অবাস্তবভাবে বাড়িয়ে দিয়েছে।
3. **ডেটা ড্রিফট চেক করুন:** প্রোডাকশনের ইনপুট ডেটার ডিস্ট্রিউশন ট্রেনিং ডেটার সাথে তুলনা করুন — বাস্তব ডেটা ট্রেনিং-এর পর থেকে বদলে গিয়ে থাকতে পারে।
4. **পাইপলাইন চেক করুন:** ফিচার ইঞ্জিনিয়ারিং, মিসিং ভ্যালু হ্যান্ডলিং, বা প্রিপ্রসেসিং ধাপে বাগ আছে কিনা দেখুন, যা ইনফারেন্সের সময় ভিন্নভাবে আচরণ করতে পারে (যেমন পুরনো ফিচার, নীরবে ব্যর্থ হওয়া join)।
5. **লেবেল/ফিডব্যাক লুপ চেক করুন:** প্রোডাকশনের গ্রাউন্ডট্রুথ লেবেল (যদি থাকে) ট্রেনিং লেবেলের মতোই একইভাবে গণনা হচ্ছে কিনা নিশ্চিত করুন।
6. **Latency/timeout সমস্যা চেক করুন:** কখনো কখনো মডেল সিস্টেম/ইনফ্রা সমস্যার কারণে আংশিক বা ডিফল্ট ইনপুট পায়, মডেলিং ভুলের কারণে নয়।
7. **ব্যর্থতা সেগমেন্ট করুন:** ইউজার সেগমেন্ট, সময়, বা ভৌগোলিক অবস্থান অনুযায়ী error ভেঙে দেখুন, যাতে বোঝা যায় গ্যাপটা কোথায় ও কখন শুরু হয়েছে — এটি প্রায়ই মূল কারণ সরাসরি চিহ্নিত করে।

22 What is the difference between data drift and concept drift?

ডেটা ড্রিফট এবং কনসেপ্ট ড্রিফট-এর মধ্যে পার্থক্য কী?

ENGLISH

1. **Data drift (covariate shift):** The distribution of input features $P(X)$ changes over time, but the relationship between features and target $P(Y|X)$ stays the same. Example: customer age distribution shifts as a product becomes popular with a younger audience.
2. **Concept drift:** The relationship between input and output $P(Y|X)$ itself changes over time, even if input distribution stays similar. Example: what counts as "fraudulent behavior" changes as fraudsters adapt their tactics.
3. **Why the distinction matters:** Data drift may only need re-calibration or retraining with fresh data; concept drift means the underlying patterns the model learned are no longer valid, often requiring new features, a new model, or a fundamentally updated approach.
4. **Detection:** Monitor input feature distributions (e.g., PSI, KL divergence) for data drift; monitor model performance/error rates over time against fresh ground truth for concept drift.

বাংলা

1. **ডেটা ড্রিফট (covariate shift):** সময়ের সাথে ইনপুট ফিচারের ডিস্ট্রিবিউশন $P(X)$ বদলে যায়, কিন্তু ফিচার ও টার্গেটের মধ্যে সম্পর্ক $P(Y|X)$ একই থাকে। উদাহরণ: একটি প্রোডাক্ট তরুণ ব্যবহারকারীদের মধ্যে জনপ্রিয় হলে গ্রাহকদের বয়সের ডিস্ট্রিবিউশন বদলে যায়।
2. **কনসেপ্ট ড্রিফট:** ইনপুট ডিস্ট্রিবিউশন একই থাকলেও, ইনপুট ও আউটপুটের সম্পর্ক $P(Y|X)$ নিজেই সময়ের সাথে বদলে যায়। উদাহরণ: প্রতারণা তাদের কৌশল পরিবর্তন করলে "প্রতারণামূলক আচরণ" কী তা বদলে যায়।
3. **পার্থক্যটা কেন গুরুত্বপূর্ণ:** ডেটা ড্রিফটে হয়তো শুধু নতুন ডেটা দিয়ে re-calibration বা retraining যথেষ্ট; কনসেপ্ট ড্রিফটের অর্থ মডেল যে প্যাটার্ন শিখেছিল তা আর বৈধ নয়, প্রায়ই নতুন ফিচার, নতুন মডেল, বা মৌলিকভাবে পরিবর্তিত পদ্ধতি দরকার হয়।
4. **শনাক্তকরণ:** ডেটা ড্রিফটের জন্য ইনপুট ফিচার ডিস্ট্রিবিউশন মনিটর করুন (যেমন PSI, KL divergence); কনসেপ্ট ড্রিফটের জন্য সময়ের সাথে মডেলের পারফরম্যান্স/error rate নতুন গ্রাউন্ডট্রুথের বিপরীতে মনিটর করুন।

23 What should be monitored after an ML model is deployed?

একটি এমএল মডেল ডিপ্লয় করার পর কী কী মনিটর করা উচিত?

ENGLISH

1. **Model performance metrics:** Accuracy, precision/recall, AUC, or business KPIs, computed against ground truth as it becomes available.
2. **Prediction distribution:** Monitor the distribution of predicted scores/classes over time to catch sudden shifts.
3. **Input data quality & drift:** Monitor feature distributions, missing value rates, schema changes, and outliers in incoming data.
4. **System health:** Latency, throughput, error rates, resource usage (CPU/GPU/memory) of the serving infrastructure.
5. **Business impact metrics:** Downstream KPIs the model is meant to influence (e.g., conversion rate, fraud losses, churn rate) — sometimes the model metric looks fine, but the business impact doesn't hold.
6. **Fairness/bias metrics:** For sensitive applications, monitor performance across different demographic segments to catch disparate impact.
7. **Alerting:** Set thresholds and alerts for all of the above so issues are caught proactively, not discovered after damage is done.

বাংলা

1. **মডেল পারফরম্যান্স মেট্রিক:** Accuracy, precision/recall, AUC, বা ব্যবসায়িক KPI, গ্রাউন্ডট্রুথ পাওয়া মাত্রই তার বিপরীতে হিসাব করা।
2. **প্রেডিকশন ডিস্ট্রিবিউশন:** সময়ের সাথে প্রেডিক্টেড স্কোর/ক্লাসের ডিস্ট্রিবিউশন মনিটর করে হঠাৎ পরিবর্তন ধরা।
3. **ইনপুট ডেটার গুণমান ও ড্রিফট:** আসা ডেটাতে ফিচার ডিস্ট্রিবিউশন, মিসিং ভ্যালুর হার, স্কিমা পরিবর্তন, এবং আউটলিয়ার মনিটর করা।
4. **সিস্টেম হেলথ:** সার্ভিং ইনফ্রাস্ট্রাকচারের latency, throughput, error rate, রিসোর্স ব্যবহার (CPU/GPU/মেমরি)।
5. **ব্যবসায়িক প্রভাব মেট্রিক:** মডেল যে ডাউনস্ট্রিম KPI-কে প্রভাবিত করার কথা (যেমন conversion rate, ফ্রড লস, churn rate) — কখনো কখনো মডেল মেট্রিক ঠিক থাকলেও ব্যবসায়িক প্রভাব মেলে না।
6. **ফেয়ারনেস/বায়াস মেট্রিক:** সংবেদনশীল অ্যাপ্লিকেশনের জন্য, বিভিন্ন ডেমোগ্রাফিক সেগমেন্টে পারফরম্যান্স মনিটর করে অসম প্রভাব ধরা।
7. **অ্যালারটিং:** উপরের সবকিছুর জন্য থ্রেশহোল্ড ও অ্যালার্ট সেট করা, যাতে সমস্যা আগেভাগেই ধরা পড়ে, ক্ষতি হওয়ার পরে নয়।

24 How would you design an end-to-end real-time fraud-detection system?

একটি এন্ড-টু-এন্ড রিয়েল-টাইম ফ্রড-ডিটেকশন সিস্টেম কীভাবে ডিজাইন করবেন?

ENGLISH

1. **Data ingestion:** Stream transaction events in real time (e.g., via Kafka/Kinesis) from payment systems.
2. **Feature engineering:** Compute real-time features (transaction velocity, amount vs. historical average, device/IP reputation) using a low-latency feature store, alongside precomputed batch features (historical aggregates).
3. **Model serving:** Deploy a lightweight, low-latency model (or ensemble) behind a scoring service that can respond within milliseconds, since fraud decisions often must happen before a transaction completes.
4. **Decisioning layer:** Combine model score with business rules (hard rules for known fraud patterns) and a calibrated threshold; route borderline cases to manual review queues.
5. **Feedback loop:** Capture confirmed fraud/non-fraud outcomes (from chargebacks, manual review, customer disputes) to continually retrain and improve the model.
6. **Handling imbalance:** Since fraud is rare, use appropriate resampling/class weighting and metrics (precision/recall, PR-AUC) rather than accuracy.
7. **Monitoring & adaptation:** Continuously monitor for concept drift (fraud patterns evolve fast), model performance, and false-positive rate impact on genuine customers; support fast model updates/retraining cycles.
8. **Explainability & compliance:** Provide reason codes for flagged transactions, both for manual reviewers and regulatory requirements.

বাংলা

1. **ডেটা ইনজেশন:** পেমেন্ট সিস্টেম থেকে রিয়েল-টাইম ট্রানজেকশন ইভেন্ট স্ট্রিম করা (যেমন Kafka/Kinesis-এর মাধ্যমে)।
2. **ফিচার ইঞ্জিনিয়ারিং:** low-latency ফিচার স্টোর ব্যবহার করে রিয়েল-টাইম ফিচার ট্রানজেকশন ভেলোসিটি, ঐতিহাসিক গড়ের তুলনায় পরিমাণ, ডিভাইস/IP রেপুটেশন) হিসাব করা, সাথে আগে থেকে হিসাব করা batch ফিচার (ঐতিহাসিক অ্যাগ্রিগেট)।
3. **মডেল সার্ভিং:** একটি হালকা, low-latency মডেল (বা এনসেম্বল) একটি স্কোরিং সার্ভিসের পেছনে ডিপ্লয় করা যা মিলিসেকেন্ডের মধ্যে সাদা দিতে পারে, কারণ ট্রানজেকশন সম্পন্ন হওয়ার আগেই ফ্রড সিদ্ধান্ত নিতে হয়।
4. **ডিসিশনিং লেয়ার:** মডেল স্কোরকে ব্যবসায়িক নিয়ম (পরিচিত ফ্রড প্যাটার্নের জন্য হার্ড রুল) এবং একটি ক্যালিব্রেটেড থ্রেশহোল্ডের সাথে মিলিয়ে সিদ্ধান্ত নেওয়া; সীমারেখার কেস ম্যানুয়াল রিভিউ কিউতে পাঠানো।
5. **ফিডব্যাক লুপ:** নিশ্চিত ফ্রড/নন-ফ্রড ফলাফল (chargeback, ম্যানুয়াল রিভিউ, গ্রাহক অভিযোগ থেকে) সংগ্রহ করে ক্রমাগত মডেল retrain ও উন্নত করা।
6. **ইমব্যালান্স হ্যান্ডলিং:** যেহেতু ফ্রড বিরল, তাই accuracy না ব্যবহার করে উপযুক্ত রিস্যাম্পলিং/ক্লাস ওয়েটিং এবং মেট্রিক্স (precision/recall, PR-AUC) ব্যবহার করা।
7. **মনিটরিং ও অভিযোজন:** কনসেপ্ট ড্রিফট (ফ্রড প্যাটার্ন দ্রুত পরিবর্তিত হয়), মডেল পারফরম্যান্স, এবং প্রকৃত গ্রাহকদের উপর false-positive rate-এর প্রভাব ক্রমাগত মনিটর করা; দ্রুত মডেল আপডেট/retraining সাইকেল সমর্থন করা।
8. **ব্যাখ্যাযোগ্যতা ও কমপ্লায়েন্স:** ফ্ল্যাগ করা ট্রানজেকশনের জন্য কারণ কোড দেওয়া, ম্যানুয়াল রিভিউয়ার এবং নিয়ন্ত্রক প্রয়োজনীয়তা — উভয়ের জন্যই।

25 How would you reduce inference latency for a production model?

প্রোডাকশন মডেলের ইনফারেন্স লেটেন্সি কীভাবে কমাবেন?

ENGLISH

1. **Model-level optimization:** Use a smaller/simpler model, quantization (reducing numerical precision, e.g., FP32 → INT8), pruning unnecessary weights, or knowledge distillation (training a small "student" model to mimic a large "teacher").
2. **Efficient serving infrastructure:** Use optimized inference runtimes (ONNX Runtime, TensorRT, TorchScript), batch multiple requests together where latency budget allows, and use hardware accelerators (GPU/TPU) if beneficial.
3. **Caching:** Cache predictions for frequently repeated inputs to avoid recomputation.
4. **Feature computation speed:** Precompute and store expensive features ahead of time (in a feature store) rather than computing them on the fly during inference.
5. **Asynchronous/parallel processing:** Parallelize independent parts of the pipeline (e.g., feature fetching from multiple sources) rather than doing everything sequentially.
6. **Reduce model complexity in the pipeline:** Simplify overly complex ensembles if a slightly less accurate but much faster single model meets business requirements.

বাংলা

1. **মডেল-স্তরের অপটিমাইজেশন:** ছোট/সহজ মডেল ব্যবহার করা, quantization (numerical precision কমানো, যেমন FP32 → INT8), অপ্রয়োজনীয় weight prune করা, বা knowledge distillation (বড় "teacher" মডেল অনুকরণ করতে ছোট "student" মডেল তৈরি করা)।
2. **দক্ষ সার্ভিং ইনফ্রাস্ট্রাকচার:** অপটিমাইজড ইনফারেন্স রানটাইম ব্যবহার করা (ONNX Runtime, TensorRT, TorchScript), লেটেন্সি বাজেট অনুমতি দিলে একাধিক রিকোয়েস্ট একসাথে batch করা, এবং উপযোগী হলে হার্ডওয়্যার অ্যাক্সিলারেটর (GPU/TPU) ব্যবহার করা।
3. **ক্যাশিং:** বারবার আসা একই ইনপুটের প্রেডিকশন ক্যাশ করে পুনরায় হিসাব এড়ানো।
4. **ফিচার গণনার গতি:** ইনফারেন্সের সময় হিসাব না করে ব্যয়বহুল ফিচার আগে থেকেই হিসাব করে (feature store-এ) সংরক্ষণ করা।
5. **Asynchronous/সমান্তরাল প্রসেসিং:** পুরো পাইপলাইন ধারাবাহিকভাবে না করে, পাইপলাইনের স্বাধীন অংশগুলো (যেমন একাধিক সোর্স থেকে ফিচার আনা) সমান্তরালে করা।
6. **পাইপলাইনে মডেল কমপ্লেক্সিটি কমানো:** সামান্য কম accurate কিন্তু অনেক দ্রুত একটি মডেলই যদি ব্যবসায়িক প্রয়োজন পূরণ করে, তাহলে অতিরিক্ত জটিল এনসেম্বল সহজ করা।

26 What is a feature store, and what problem does it solve?

ফিচার স্টোর কী, এবং এটি কী সমস্যার সমাধান করে?

ENGLISH

1. **Definition:** A centralized system for storing, managing, and serving machine learning features consistently across training and serving/production environments.
2. **Problem it solves — training/serving skew:** Without a feature store, teams often compute features differently for offline training (e.g., in a batch Spark job) vs. online serving (e.g., in a real-time API), causing subtle bugs and performance gaps.
3. **Problem it solves — reusability:** Different teams/models often need the same underlying features (e.g., "user's average order value"); a feature store avoids duplicated, inconsistent computation across teams.
4. **Problem it solves — point-in-time correctness:** Ensures that when building training data, features reflect only the information that was actually available at that point in time, preventing leakage from future data.
5. **Components:** Typically has an offline store (for training, batch features) and an online store (low-latency lookups for real-time serving), with a shared feature definition/registry. Examples: Feast, Tecton, SageMaker Feature Store.

বাংলা

1. **সংজ্ঞা:** ট্রেনিং এবং সার্ভিং/প্রোডাকশন পরিবেশে সামঞ্জস্যপূর্ণভাবে মেশিন লার্নিং ফিচার সংরক্ষণ, ব্যবস্থাপনা, এবং সার্ভ করার একটি কেন্দ্রীকৃত সিস্টেম।
2. **যে সমস্যার সমাধান করে — ট্রেনিং/সার্ভিং স্কিউ:** ফিচার স্টোর ছাড়া, টিমগুলো অফলাইন ট্রেনিং (যেমন batch Spark job-এ) এবং অনলাইন সার্ভিং (যেমন রিয়েল-টাইম API-তে)-এর জন্য প্রায়ই আলাদাভাবে ফিচার হিসাব করে, যা সূক্ষ্ম বাগ ও পারফরম্যান্স গ্যাপ তৈরি করে।
3. **যে সমস্যার সমাধান করে — পুনর্ব্যবহারযোগ্যতা:** বিভিন্ন টিম/মডেলের প্রায়ই একই মূল ফিচার দরকার হয় (যেমন "ইউজারের গড় অর্ডার ভ্যালু"); ফিচার স্টোর টিমগুলোর মধ্যে ডুপ্লিকেট, অসামঞ্জস্যপূর্ণ হিসাব এড়ায়।
4. **যে সমস্যার সমাধান করে — point-in-time সঠিকতা:** নিশ্চিত করে যে ট্রেনিং ডেটা তৈরির সময়, ফিচারগুলো শুধু সেই সময়ে বাস্তবেই পাওয়া তথ্যই প্রতিফলিত করে, ফলে ভবিষ্যতের ডেটা থেকে লিকেজ প্রতিরোধ হয়।
5. **উপাদান:** সাধারণত একটি অফলাইন স্টোর (ট্রেনিং, batch ফিচারের জন্য) এবং একটি অনলাইন স্টোর (রিয়েল-টাইম সার্ভিং-এর জন্য low-latency lookup) থাকে, সাথে একটি শেয়ার্ড ফিচার সংজ্ঞা/রেজিস্ট্রিউদাহরণ: Feast, Tecton, SageMaker Feature Store।

27 How would you version data, code, features, and models for reproducibility?

রিপ্রোডিউসিবিলিটির জন্য ডেটা, কোড, ফিচার এবং মডেল কীভাবে ভার্সন করবেন?

ENGLISH

1. **Code versioning:** Use Git for all training/inference code, with tagged releases corresponding to specific model versions.
2. **Data versioning:** Use tools like DVC, LakeFS, or Delta Lake to version datasets, so any training run can be traced back to the exact data snapshot used.
3. **Feature versioning:** Track feature definitions and their transformation logic in a feature store or registry, with clear versioning when definitions change.
4. **Model versioning:** Use a model registry (MLflow, SageMaker Model Registry, Vertex AI Model Registry) to store trained model artifacts along with metadata: training data version, code commit hash, hyperparameters, and evaluation metrics.
5. **Experiment tracking:** Use tools like MLflow or Weights & Biases to log every experiment's configuration and results, enabling comparison and reproducibility.
6. **Environment reproducibility:** Use containerization (Docker) and dependency lock files (requirements.txt with pinned versions, conda-lock) to freeze the exact software environment.

বাংলা

1. **কোড ভার্সনিং:** সব ট্রেনিং/ইনফারেন্স কোডের জন্য Git ব্যবহার করা, নির্দিষ্ট মডেল ভার্সনের সাথে মিলিয়ে ট্যাগড রিলিজ রাখা।
2. **ডেটা ভার্সনিং:** DVC, LakeFS, বা Delta Lake-এর মতো টুল ব্যবহার করে ডেটাসেট ভার্সন করা, যাতে যেকোনো ট্রেনিং রান ঠিক কোন ডেটা স্ন্যাপশট ব্যবহার করেছিল তা খুঁজে বের করা যায়।
3. **ফিচার ভার্সনিং:** ফিচার স্টোর বা রেজিস্ট্রি ফিচারের সংজ্ঞা ও তাদের ট্রান্সফরমেশন লজিক ট্রাক করা, সংজ্ঞা বদলালে স্পষ্ট ভার্সনিং রাখা।
4. **মডেল ভার্সনিং:** ট্রেন করা মডেল আর্টিফ্যাক্ট মেটাডেটাসহ (ট্রেনিং ডেটা ভার্সন, কোড কমিট হ্যাশ, হাইপারপ্যারামিটার, মূল্যায়ন মেট্রিক) সংরক্ষণ করতে একটি মডেল রেজিস্ট্রি ব্যবহার করা (MLflow, SageMaker Model Registry, Vertex AI Model Registry)।
5. **এক্সপেরিमेंট ট্র্যাকিং:** প্রতিটি এক্সপেরিमेंটের কনফিগারেশন ও ফলাফল লগ করতে MLflow বা Weights & Biases-এর মতো টুল ব্যবহার করা, যাতে তুলনা ও রিপ্রোডিউসিবিলিটি সহজ হয়।
6. **পরিবেশের রিপ্রোডিউসিবিলিটি:** সঠিক সফটওয়্যার পরিবেশ স্থির রাখতে কন্টেইনারাইজেশন (Docker) এবং dependency lock ফাইল (নির্দিষ্ট ভার্সনসহ requirements.txt, conda-lock) ব্যবহার করা।

28 What is the difference between batch inference and online inference?

ব্যাচ ইনফারেন্স এবং অনলাইন ইনফারেন্সের মধ্যে পার্থক্য কী?

ENGLISH

1. **Batch inference:** Predictions are generated for a large set of inputs at once, on a schedule (e.g., nightly), and stored for later use. No strict latency requirement per prediction; optimized for throughput.
2. **Online inference:** Predictions are generated on-demand, in real time, typically in response to a single request (e.g., a user visiting a webpage). Requires low latency (often milliseconds), and the serving system must always be available.
3. **Trade-offs:** Batch is simpler, cheaper, easier to scale and debug, but predictions can be stale by the time they're used. Online is fresher/more responsive but requires more robust, always-on infrastructure and careful latency optimization.
4. **When to use which:** Batch for use cases like periodic churn scoring, monthly recommendations, or offline reporting. Online for use cases like fraud detection at checkout, real-time recommendations, or search ranking.

বাংলা

1. **ব্যাচ ইনফারেন্স:** একসাথে অনেক ইনপুটের জন্য প্রেডিকশন তৈরি করা হয়, একটি নির্দিষ্ট সময়সূচি অনুযায়ী (যেমন প্রতি রাতে), এবং পরে ব্যবহারের জন্য সংরক্ষণ করা হয়। প্রতিটি প্রেডিকশনের জন্য কঠোর latency প্রয়োজন নেই; throughput-এর জন্য অপটিমাইজড।
2. **অনলাইন ইনফারেন্স:** চাহিদা অনুযায়ী, রিয়েল-টাইমে প্রেডিকশন তৈরি হয়, সাধারণত একটি একক রিকোয়েস্টের জবাবে (যেমন কোনো ইউজার একটি ওয়েবপেজ ভিজিট করলে)। কম latency দরকার (প্রায়ই মিলিসেকেন্ড), এবং সার্ভিং সিস্টেম সবসময় উপলব্ধ থাকতে হবে।
3. **ট্রেড-অফ:** ব্যাচ সহজ, সাশ্রয়ী, স্কেল ও ডিবাগ করা সহজ, কিন্তু ব্যবহারের সময় প্রেডিকশন পুরনো হয়ে যেতে পারে। অনলাইন বেশি তাজা/রেসপন্সিভ কিন্তু বেশি robust, সবসময়-চালু ইনফ্রাস্ট্রাকচার এবং সতর্ক latency অপটিমাইজেশন দরকার হয়।
4. **কখন কোনটা ব্যবহার করবেন:** পর্যায়ক্রমিক churn স্কোরিং, মাসিক সুপারিশ, বা অফলাইন রিপোর্টিং-এর মতো ক্ষেত্রে ব্যাচ। চেকআউটে ফ্রড ডিটেকশন, রিয়েল-টাইম সুপারিশ, বা সার্চ র‍্যাঙ্কিং-এর মতো ক্ষেত্রে অনলাইন।

29 How would you safely deploy a new model version?

একটি নতুন মডেল ভার্সন কীভাবে নিরাপদে ডিপ্লয় করবেন?

ENGLISH

1. **Offline validation first:** Thoroughly test the new model on held-out and recent data before considering deployment; compare against the current production model's metrics.
2. **Shadow deployment:** Run the new model alongside the current one in production, on live traffic, without affecting real decisions, and compare outputs to catch issues before it impacts users.
3. **Canary release:** Roll out the new model to a small percentage of traffic (e.g., 5%) first, closely monitor performance and business metrics, then gradually increase the traffic share if it performs well.
4. **A/B testing:** Run a controlled experiment comparing the new model vs. the old model on randomly split user groups, measuring impact on both model metrics and business KPIs.
5. **Rollback plan:** Ensure there's an easy, fast way to revert to the previous model version if problems are detected.
6. **Monitoring during rollout:** Watch latency, error rates, prediction distributions, and business metrics closely during and after the rollout.

বাংলা

1. **প্রথমে অফলাইন যাচাই:** ডিপ্লয়মেন্ট বিবেচনা করার আগে নতুন মডেলকে held-out এবং সাম্প্রতিক ডেটাতে ভালোভাবে টেস্ট করা; বর্তমান প্রোডাকশন মডেলের মেট্রিকের সাথে তুলনা করা।
2. **শ্যাডো ডিপ্লয়মেন্ট:** নতুন মডেলকে প্রোডাকশনে, লাইভ ট্রাফিকে, বর্তমান মডেলের পাশাপাশি চালানো, আসল সিদ্ধান্তে প্রভাব না ফেলে, এবং ইউজারের উপর প্রভাব পড়ার আগেই আউটপুট তুলনা করে সমস্যা ধরা।
3. **ক্যানারি রিলিজ:** প্রথম ট্রাফিকের একটি ছোট অংশে (যেমন 5%) নতুন মডেল চালু করা, পারফরম্যান্স ও ব্যবসায়িক মেট্রিক ভালোভাবে মনিটর করা, তারপর ভালো পারফর্ম করলে ধীরে ধীরে ট্রাফিকের অংশ বাড়ানো।
4. **A/B টেস্টিং:** র্যান্ডমভাবে ভাগ করা ইউজার গ্রুপে নতুন মডেল বনাম পুরনো মডেল তুলনা করে একটি নিয়ন্ত্রিত এক্সপেরিমেন্ট চালানো, মডেল মেট্রিক এবং ব্যবসায়িক KPI — উভয়ের উপর প্রভাব মাপা।
5. **রোলব্যাক পরিকল্পনা:** সমস্যা ধরা পড়লে আগের মডেল ভার্সনে দ্রুত ও সহজে ফিরে যাওয়ার ব্যবস্থা রাখা।
6. **রোলআউটের সময় মনিটরিং:** রোলআউটের সময় এবং পরে latency, error rate, প্রেডিকশন ডিস্ট্রিবিউশন, এবং ব্যবসায়িক মেট্রিক ভালোভাবে নজরে রাখা।

30 Describe one ML project from problem definition through deployment and monitoring.

সমস্যা নির্ধারণ থেকে ডিপ্লয়মেন্ট ও মনিটরিং পর্যন্ত একটি এমএল প্রজেক্ট বর্ণনা করুন।

ENGLISH

Walk through a real (or realistic) end-to-end project using this structure, tailored to your own experience:

1. **Problem definition:** What business problem were you solving, and why was ML the right approach? What was the success metric (both model metric and business metric)?
2. **Data collection & exploration:** Where did data come from? What did EDA reveal (missing values, imbalances, outliers, correlations)?
3. **Feature engineering:** What features did you create, and why were they informative for the problem?
4. **Modeling:** What model(s) did you try, why, and how did you validate them (cross-validation, proper train/val/test splits)?
5. **Evaluation:** What metrics did you use and why? How did you make sure the offline evaluation reflected the real business goal?
6. **Deployment:** How was the model served (batch or online)? What deployment strategy did you use (canary, A/B test)?
7. **Monitoring & iteration:** What did you monitor post-deployment? Did you observe drift, and how did you respond? What would you improve next?

Interview tip: Structure your answer using this framework, be specific with numbers and decisions you made, and be honest about trade-offs and mistakes — that shows real, practical experience rather than a textbook answer.

বাংলা

নিজের অভিজ্ঞতা অনুযায়ী একটি বাস্তব (বা বাস্তবসম্মত) এন্ড-টু-এন্ড প্রজেক্ট এই কাঠামো ব্যবহার করে বর্ণনা করুন:

1. **সমস্যা নির্ধারণ:** কোন ব্যবসায়িক সমস্যা সমাধান করছিলেন, এবং কেন ML-ই সঠিক পদ্ধতি ছিল? সাফল্যের মেক্সিকী ছিল (মডেল মেক্সিকী এবং ব্যবসায়িক মেক্সিকী — দুটোই)?
2. **ডেটা সংগ্রহ ও এক্সপ্লোরেশন:** ডেটা কোথা থেকে এসেছিল? EDA থেকে কী পাওয়া গিয়েছিল (মিসিং ভ্যালু, ইমব্যালান্স, আউটলায়ার, correlation)?
3. **ফিচার ইঞ্জিনিয়ারিং:** কী কী ফিচার তৈরি করেছিলেন, এবং সেগুলো সমস্যার জন্য কেন তথ্যবহুল ছিল?
4. **মডেলিং:** কোন মডেল(গুলো) চেষ্টা করেছিলেন, কেন, এবং কীভাবে সেগুলো যাচাই করেছিলেন (cross-validation, সঠিক train/val/test split)?
5. **মূল্যায়ন:** কী মেক্সিকী ব্যবহার করেছিলেন এবং কেন? অফলাইন মূল্যায়ন যে আসল ব্যবসায়িক লক্ষ্য প্রতিফলিত করছে তা কীভাবে নিশ্চিত করেছিলেন?
6. **ডিপ্লয়মেন্ট:** মডেল কীভাবে সার্ভ করা হয়েছিল (ব্যাচ নাকি অনলাইন)? কী ডিপ্লয়মেন্ট কৌশল ব্যবহার করেছিলেন (ক্যানারি, A/B টেস্ট)?
7. **মনিটরিং ও পুনরাবৃত্তি:** ডিপ্লয়মেন্টের পর কী মনিটর করেছিলেন? ড্রিফট লক্ষ্য করেছিলেন কিনা, এবং কীভাবে প্রতিক্রিয়া জানিয়েছিলেন? এরপর কী উন্নত করতেন?

ইন্টারভিউ টিপ: এই কাঠামো ব্যবহার করে উত্তর সাজান, নিজের নেওয়া সিদ্ধান্ত ও সংখ্যা নিয়ে নির্দিষ্ট থাকুন, এবং ডে-অফ ও ভুল সম্পর্কে সৎ থাকুন — এতে বোঝা যায় এটি একটি বাস্তব, প্র্যাকটিক্যাল অভিজ্ঞতা, শুধু বইয়ের উত্তর নয়।